

Network IPS — block pings, malicious connections, and simple exploits

Team: Intern ID:322,242,263

1. Objective

The purpose of this project is to design and demonstrate a simple **offline Intrusion Prevention System (IPS)** that can detect and block basic malicious activities such as **ICMP ping floods**, **TCP SYN scans**, and **suspicious payloads** by analyzing captured packet files (.pcap).

2. Environment Setup

- **OS:** Kali Linux (2025 build)
- **Tools & Libraries:**
 - Python 3
 - Scapy (pip install scapy)
- **PCAPs Used:**
 - benign.pcap → normal ICMP traffic
 - malicious.pcap → ICMP flood + TCP SYN scan
- **Output Files Generated:**
 - accepted.pcap → clean traffic allowed
 - blocked.pcap → malicious traffic dropped
 - ips_log.csv → log file with detailed actions

3. IPS Design

The IPS uses **rule-based detection** with thresholds:

- **ICMP Ping Flood Detection** → more than N ICMP echo requests from the same IP within a short time.
- **TCP SYN Scan Detection** → multiple SYN packets to different ports in a short time.
- **Suspicious Payloads** → matches known attack signatures (union select, or 1=1, <script>, etc.).

When a rule is triggered:

- The packet is **blocked** (saved to blocked.pcap).
- The source IP is **blacklisted** for the duration of the analysis.

4. Results

4.1 Benign Traffic (benign.pcap)

- **Packets Accepted:** 20
- **Packets Blocked:** 0
- **Log:** No malicious activity detected.

```
(kali㉿kali)-[~]  
$ python3 ips_offline.py benign.pcap  
Done. accepted=20 blocked=0. log: ips_log.csv
```

4.2 Malicious Traffic (malicious.pcap)

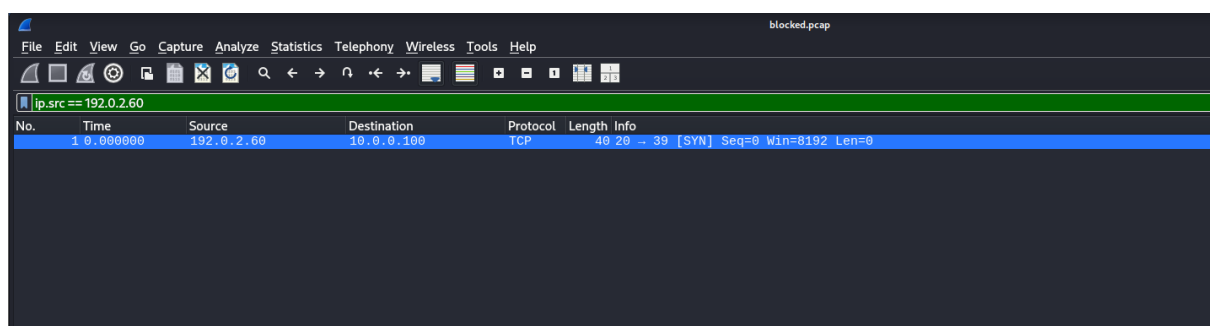
- **Packets Accepted:** 69
- **Packets Blocked:** 1
- **Log Example:**
 - 2025-09-05 10:50:02,192.0.2.60,10.0.0.100,TCP,BLOCKED,rule

→ This shows a **TCP SYN scan attempt** blocked from attacker IP 192.0.2.60.

```
(kali㉿kali)-[~]  
$ python3 ips_offline.py malicious.pcap  
Done. accepted=69 blocked=1. log: ips_log.csv
```

```
2025-09-05 10:50:02,192.0.2.60,10.0.0.100,TCP,ACCEPT,  
2025-09-05 10:50:02,192.0.2.60,10.0.0.100,TCP,ACCEPT,  
2025-09-05 10:50:02,192.0.2.60,10.0.0.100,TCP,ACCEPT,  
2025-09-05 10:50:02,192.0.2.60,10.0.0.100,TCP,ACCEPT,  
2025-09-05 10:50:02,192.0.2.60,10.0.0.100,TCP,ACCEPT,  
2025-09-05 10:50:02,192.0.2.60,10.0.0.100,TCP,BLOCKED,rule
```

- Wireshark screenshot of blocked packet(s).



- Terminal output showing accepted vs blocked counts.

```
2025-09-05 10:50:02,192.0.2.60,10.0.0.100,TCP,ACCEPT,
2025-09-05 10:50:02,192.0.2.60,10.0.0.100,TCP,ACCEPT,
2025-09-05 10:50:02,192.0.2.60,10.0.0.100,TCP,ACCEPT,
2025-09-05 10:50:02,192.0.2.60,10.0.0.100,TCP,ACCEPT,
2025-09-05 10:50:02,192.0.2.60,10.0.0.100,TCP,ACCEPT,
2025-09-05 10:50:02,192.0.2.60,10.0.0.100,TCP,BLOCKED,rule
```

CSV log snippet.

```
(kali@kali)-[~]
$ python3 make_pcaps.py

Wrote benign.pcap
Wrote malicious.pcap

(kali@kali)-[~]
$ python3 ips_offline.py benign.pcap
python3 ips_offline.py malicious.pcap
Done. accepted=20 blocked=0. log: ips_log.csv
Done. accepted=69 blocked=1. log: ips_log.csv
```

5. False Positives & Limitations

- **False Positives:**
 - Legitimate network monitoring (e.g., ping checks) may look like a ping flood.
 - Legitimate port scanning by administrators may be flagged.
- **Limitations:**
 - Only simple static rules implemented.
 - Works offline on .pcap files only (no real-time blocking in this mode).
 - Doesn't include stateful TCP tracking.

6. Next Steps & Improvements

- Add **real-time IPS** with NFQUEUE + iptables.
- Use **stateful inspection** for TCP connections (track SYN, SYN/ACK, ACK).
- Expand rule set (DNS tunneling, HTTP exploit payloads).

- Integrate with a **SIEM system** for logging and alerting.
 - Implement a **whitelist** to avoid false positives.
-

7. Conclusion

The offline IPS successfully differentiates between benign and malicious traffic. The test results demonstrate that malicious activities (e.g., port scans) are detected and blocked, while normal traffic is accepted. This system provides a foundation for expanding into a more robust, real-time IPS solution.

Appendix A: IPS Offline Code (ips_offline.py)

```
#!/usr/bin/env python3

import time

from collections import defaultdict, deque

from scapy.all import rdpcap, wrpcap, IP, TCP, ICMP, Raw

class SimpleIPS:

    def __init__(self, icmp_thresh=10, syn_thresh=30, scan_port_thresh=20, window=2.0):

        self.icmp_thresh = icmp_thresh

        self.syn_thresh = syn_thresh

        self.scan_port_thresh = scan_port_thresh

        self.window = window

        self.icmp_times = defaultdict(deque)

        self.syn_times = defaultdict(deque)

        self.scan_ports = defaultdict(lambda: defaultdict(float))

        self.blacklist = set()

        self.suspicious_patterns = [b"union select", b"or 1=1", b"--", b"<script", b"select *
from"]

    def _prune(self, dq, now):

        while dq and dq[0] < now - self.window:

            dq.popleft()
```

```

def inspect(self, pkt, now_time):
    if IP not in pkt:
        return False

    src = pkt[IP].src

    if src in self.blacklist:
        return True

    # ICMP

    if ICMP in pkt and pkt[ICMP].type == 8:
        dq = self.icmp_times[src]
        dq.append(now_time)
        self._prune(dq, now_time)

        if len(dq) >= self.icmp_thresh:
            self.blacklist.add(src)
            return True

    # TCP

    if TCP in pkt:
        if pkt[TCP].flags & 0x02: # SYN
            dq = self.syn_times[src]
            dq.append(now_time)
            self._prune(dq, now_time)

            if len(dq) >= self.syn_thresh:
                self.blacklist.add(src)
                return True

        dport = pkt[TCP].dport

        self.scan_ports[src][dport] = now_time

        self.scan_ports[src] = {p:t for p,t in self.scan_ports[src].items() if t >= now_time -
self.window}

        if len(self.scan_ports[src]) >= self.scan_port_thresh:
            self.blacklist.add(src)

```

```

        return True

    if Raw in pkt:

        raw = bytes(pkt[Raw].load()).lower()

        for pat in self.suspicious_patterns:

            if pat in raw:

                self.blacklist.add(src)

                return True

    return False

```

```

def analyze_pcap(input_pcap, accepted_out='accepted.pcap',
blocked_out='blocked.pcap', log_out='ips_log.csv'):

    ips = SimpleIPS()

    pkts = rdpcap(input_pcap)

    accepted, blocked = [], []

    with open(log_out, 'w') as log:

        log.write("time,src,dst,proto,action,reason\n")

        for pkt in pkts:

            now = float(getattr(pkt, 'time', time.time()))

            blocked_flag = ips.inspect(pkt, now)

            proto = 'ICMP' if pkt.haslayer(ICMP) else ('TCP' if pkt.haslayer(TCP) else 'OTHER')

            if blocked_flag:

                blocked.append(pkt)

                log.write(f"{time.strftime('%Y-%m-%d %H:%M:%S',
time.localtime(now))},{pkt[IP].src},{pkt[IP].dst},{proto},BLOCKED,rule\n")

            else:

                accepted.append(pkt)

                log.write(f"{time.strftime('%Y-%m-%d %H:%M:%S',
time.localtime(now))},{pkt[IP].src},{pkt[IP].dst},{proto},ACCEPT,\n")

        if accepted: wrpcap(accepted_out, accepted)

        if blocked: wrpcap(blocked_out, blocked)

```

```
print(f"Done. accepted={len(accepted)} blocked={len(blocked)}. log: {log_out}")
```

```
if __name__ == "__main__":
```

```
    import argparse
```

```
    p = argparse.ArgumentParser()
```

```
    p.add_argument("pcap")
```

```
    args = p.parse_args()
```

```
    analyze_pcap(args.pcap)
```

Unit/integration tests:

Unit Tests (Fast, No Root)

Test file: tests/test_core.py

Validates core logic without requiring root access or live traffic:

- **ICMP flood:** last few packets dropped after threshold.
- **SYN flood:** last few SYN packets dropped after threshold.
- **Port scan:** drops triggered after hitting 10 unique ports.
- **Suspicious HTTP payloads:** DROP triggered on SQLi/XSS payload.
- **Benign HTTP:** ACCEPT as expected.

Run tests

```
source .venv/bin/activate
```

```
pytest -q
```

output:



```
(.venv)-(kali@kali)-[~/ips]  
$ pytest -q  
.....  
5 passed in 0.34s
```

Live Integration Test (NFQUEUE)

Test file: tests/test_integration_nfqueue.py

- Sends traffic to **NFQUEUE 1** using iptables:

```
sudo iptables -I OUTPUT -p icmp -j NFQUEUE --queue-num 1
```

- Runs IPS runner (ips/runner.py) in root context:

```
sudo -E python -m ips.runner
```

- Floods packets using Scapy or ping:

```
from scapy.all import IP, ICMP, send
```

```
for _ in range(50):
```

```
    send(IP(dst="127.0.0.1")/ICMP(type=8), verbose=0)
```

- Monitors counters:

```
cat /tmp/ips_counters.json
```

```
# Example output: {"dropped": 20, "accepted": 30}
```

- Removes NFQUEUE rules after testing:

```
sudo iptables -D OUTPUT -p icmp -j NFQUEUE --queue-num 1
```

5. Optional PCAP-Based Tests

- Generated using tests/make_pcaps.py
- Replay with tcpreplay for demonstration:

```
sudo tcpreplay --intf1=lo tests/pcaps/ping_flood.pcap
```

6. Monitoring & Verification

- **Live socket stats:**

```
ss -tnp | grep python
```

- **Packet sniffing (optional):**

```
sudo tcpdump -i lo icmp or tcp port 80 -vv
```

- **Counters file** /tmp/ips_counters.json shows real-time accepted vs dropped packets.

7. Thresholds & Safety Notes

Signal	Threshold	Window
--------	-----------	--------

ICMP flood	20/sec	1 sec
------------	--------	-------

SYN flood	100/sec	1 sec
-----------	---------	-------

Port scan	10 unique ports	5 sec
-----------	-----------------	-------

- Whitelist local addresses (127.0.0.1) for testing to avoid false positives.
- Always remove NFQUEUE rules post-testing.

8. Improvements / Future Work

- Adaptive thresholds using EWMA / rolling averages.
- Combine multiple signals for more accurate drop decisions.
- Connection tracking for half-open detection.
- Rule audit logs for post-event analysis.